

WTC Connecta — Backend

API REST desenvolvida em **Spring Boot 3.2.5 + Java 17** para o sistema de CRM e mensageria corporativa WTC Connecta.

Tecnologias

- Java 17
 - Spring Boot 3.2.5
 - Spring Security + JWT (JJWT 0.12.5)
 - Spring Data MongoDB
 - MongoDB Atlas (banco em nuvem)
 - Firebase Admin SDK 9.2.0 (push notifications)
 - Maven
-

Pré-requisitos

Antes de rodar o projeto, instale:

- JDK 17+ — obrigatório (testado também com Java 24)
- IntelliJ IDEA ou qualquer IDE Java
- Conexão com internet — o banco está no MongoDB Atlas (nuvem)

! Não é necessário instalar o MongoDB localmente. O banco já está configurado na nuvem.

Configuração

1. Clonar ou extrair o projeto

```
bash

# Extraia o .zip em uma pasta de sua preferência
# Ex: C:\projetos-java\wtc-backend
```

2. Configurar o application.yml

Abra o arquivo `src/main/resources/application.yml` e configure:

```
yaml
```

```
spring:
  application:
    name: wtc-backend
  data:
    mongodb:
      uri: mongodb+srv://wtcadmin:SENHA@wtc.khkq5ro.mongodb.net/wtcdb?retryWrites=tr

server:
  port: 8080

jwt:
  secret: wtc_connecta_secret_key_minimo_256_bits_seguro
  expiration: 86400000
```

Atenção: substitua `SENHA` pela senha do banco fornecida junto com o projeto. O arquivo `firebase-adminsdk.json` já está incluído em `src/main/resources/`.

3. Rodar o projeto

Pelo IntelliJ:

- Abra a pasta do projeto
- Aguarde o Maven baixar as dependências
- Abra `Application.java` e clique em **Run**

Pelo terminal:

```
bash

# Windows
mvnw.cmd spring-boot:run

# Linux/Mac
./mvnw spring-boot:run
```

4. Verificar que subiu

No console deve aparecer:

```
Started Application in X.XXX seconds
Tomcat started on port 8080
```

Endpoints principais

Autenticação (público)

Método	Rota	Descrição
POST	/api/auth/register	Registrar novo usuário
POST	/api/auth/login	Login — retorna token JWT

Exemplo de registro:

```
json

POST http://localhost:8080/api/auth/register
{
  "name": "Operador WTC",
  "email": "op@wtc.com",
  "password": "123456",
  "role": "OPERATOR"
}
```

Exemplo de login:

```
json

POST http://localhost:8080/api/auth/login
{
  "email": "op@wtc.com",
  "password": "123456"
}
```

Todos os endpoints abaixo exigem o header: `Authorization: Bearer {token}`

Clientes (requer role OPERATOR)

Método	Rota	Descrição
POST	/api/clients	Criar cliente
GET	/api/clients	Listar todos
GET	/api/clients?name=joao	Buscar por nome
GET	/api/clients?status=ACTIVE	Filtrar por status

Método	Rota	Descrição
GET	/api/clients?tag=vip	Filtrar por tag
GET	/api/clients?minScore=80	Filtrar por score
GET	/api/clients/{id}	Buscar por ID
PUT	/api/clients/{id}	Atualizar cliente
DELETE	/api/clients/{id}	Deletar cliente
POST	/api/clients/{id}/notes	Adicionar anotação
GET	/api/clients/{id}/notes	Listar anotações

Campanhas (requer role OPERATOR)

Método	Rota	Descrição
POST	/api/campaigns	Criar campanha
GET	/api/campaigns	Listar todas
GET	/api/campaigns?status=DRAFT	Filtrar por status
GET	/api/campaigns/{id}	Buscar por ID
POST	/api/campaigns/{id}/dispatch	Disparar campanha
DELETE	/api/campaigns/{id}	Deletar campanha

Exemplo de criação e disparo:

json

```
POST http://localhost:8080/api/campaigns
{
  "title": "Evento Especial",
  "body": "Participe do nosso evento exclusivo!",
  "url": "https://wtc.com/evento",
  "actions": [
    { "action": "btn1", "title": "Inscrever-se" },
    { "action": "btn2", "title": "Saiba Mais" }
  ],
  "actionUrls": {
    "btn1": "https://wtc.com/inscricao",
    "btn2": "https://wtc.com/detalhes"
  },
  "targetTags": ["vip"]
}

POST http://localhost:8080/api/campaigns/{id}/dispatch
```

Mensagens (requer autenticação)

Método	Rota	Descrição
POST	<code>/api/messages/direct</code>	Enviar mensagem 1:1
GET	<code>/api/messages/conversation/{id}</code>	Histórico da conversa
GET	<code>/api/messages/unread</code>	Mensagens não lidas
GET	<code>/api/messages/unread/count</code>	Contagem de não lidas
PATCH	<code>/api/messages/{id}/read</code>	Marcar como lida

Integração com o App Android

O app Android consome esta API via Retrofit. Configure o `BASE_URL` conforme o ambiente:

Emulador Android (mesmo PC):

```
kotlin

private const val BASE_URL = "http://10.0.2.2:8080/"
```

Dispositivo físico (mesma rede Wi-Fi):

kotlin

```
// Descubra o IP: cmd → ipconfig → "Endereço IPv4"
private const val BASE_URL = "http://192.168.X.X:8080/"
```

Arquitetura

```
br.com.wtc/
├─ config/           # SecurityConfig, CorsConfig, MongoConfig, FirebaseConfig
├─ security/         # JwtFilter, JwtUtil, CustomUserDetailsService
├─ domain/
│   ├─ model/        # Client, User, Message, Campaign, Note, Group, Division
│   ├─ repository/   # Interfaces MongoRepository
│   └─ service/      # AuthService, ClientService, CampaignService,
MessageService, NoteService
├─ web/
│   └─ controller/   # AuthController, ClientController, CampaignController,
MessageController
│       └─ dto/       # LoginRequest/Response, ClientRequest/Response,
CampaignRequest/Response
├─ infra/            # FirebasePushService, SchedulerJob
└─ exception/        # GlobalExceptionHandler, BusinessException,
ResourceNotFoundException
```

Banco de dados — Collections MongoDB

Collection	Descrição
users	Operadores e clientes do sistema
clients	Clientes do CRM com tags, score e status
messages	Mensagens de chat e campanhas
campaigns	Campanhas com segmentação e status
notes	Anotações rápidas por cliente
groups	Grupos de clientes
divisions	Divisões/setores da empresa

Equipe

Projeto desenvolvido para o desafio WTC Connecta — FIAP 2026.